98.6
Pass

Aditya B Kakde

PDF generated at: 15 Feb 2026 04:16:07 UTC

View this report on HackerRank

Score

98.6% • 172.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 47 min 28 sec on 15 Feb 2026 08:56:25 IST

Candidate Information

Email	adityakakde11@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	15 Feb 2026 08:56:25 IST
Time taken	47 min 28 sec/ 90 min
Work Experience	> 5 years
Invited by	Kalyani
Invited on	13 Feb 2026 18:06:18 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	87%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Hard	100%	Spring Boot	100%
TDD	100%	Medium	98%
Spring	50%	Sorting	100%
Greedy Algorithms	100%	Two Pointers	100%
Greedy	100%	Java	100%

Questions

Spring Boot (Advanced) • 22.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Output Multiple Choice	Spring Boot (Advanced)	5/5	-	57 sec	✓

2	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 12 sec	
3	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	3 min 40 sec	
4	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 35 sec	
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	56 sec	

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Maximum Array Correlation Coding	Problem Solving (Intermediate)	75/75	-	12 min 47 sec	

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: DataVille's Optimal Storage Coding	Java (Intermediate)	75/75	-	24 min 38 sec	✓

1. Output

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }
```

```
    @Test
    public void test2()
    {
        logWriter.write("Entered Test 2");
        assertTrue(someClassToTest.otherMethod());
    }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ○ The output file is created but not deleted.
- ● **The output file is created and eventually deleted.** ✓
- ○ The create and delete blocks will not be executed.

No comments.

2. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- ● **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- ○ None of the above

No comments.

---

3. Spring Scopes

Partially correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] Both are used to define the scope of spring beans.
- [x] **@Scope("request") creates a bean instance for a single HTTP request.**
- [ ] @RequestScope can be used instead of @Scope("request").
- [x] **@Scope("prototype") will return a different instance every time it is requested from the container.**

No comments.

---

4. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Configuration class annotated with @EnableGlobalMethodSecurity
- Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)**
- None above

No comments.

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }
- private void manage(HttpSecurity http) {   http.csrf() }
- private void manage(HttpSecurity http) {   http.csrf().enable() }
- ** None of the above. CSRF is configured by default when Spring Security Dependency is added. **

No comments.

6. Maximum Array Correlation

Correct

Coding | Sorting | Greedy Algorithms | Medium | Two Pointers

Question description

For two arrays *a* and *b* of equal length, the array correlation is defined as the sum of all values *b[i]* where *b[i]* is greater than *a[i]*.

Given two integer arrays *a* and *b* of the same length, rearrange the elements of array *b* to maximize the array correlation. Return the maximum possible array correlation value.

Note: You are not allowed to rearrange the elements of the array *a*.

Example:

n = 5, *a* = [1, 2, 3, 4, 5], *b* = [3, 5, 4, 6, 2]

b can be rearranged to [2, 3, 4, 5, 6]. Now, for each index, the integers in *b* are greater than those in *a.* The array correlation is 2 + 3 + 4 + 5 + 6 = 20.

Function Description

Complete the function *getMaxArrayCorrelation* in the editor with the following parameter(s):

a[n]: the fixed order array

b[n]: the array to reorder

Returns:

long int: the maximum possible array correlation

Constraints

- $1 \le n \le 2 \times 10^5$
- $1 \le a[i], b[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, that represents the number of elements in *a*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *a[i].*
The next line contains an integer, *n*, that represents the number of elements in *b*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *b[i].*

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
5       →  a[] size n = 5
1       →  a = [1, 4, 2, 1, 3]
4
2
1
3
```

```
5       →  b[] size n = 5
2       →  b = [2, 3, 1, 2, 2]
3
1
2
2
```

Sample Output

```
7
```

Explanation

The second array can be rearranged to *b = [2, 1, 3, 2, 2]*. *b[i] > a[i]* for *i* = 0, 3, and 4, and the array correlation is 2 + 3 + 2 = 7.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
4       →  a[] size n = 4
1       →  a = [1, 9, 4, 2]
9
4
2
4       →  b[] size n = 4
8       →  b = [8, 4, 3, 1]
4
3
1
```

Sample Output

```
15
```

Explanation

The second array can be rearranged to *b = [3, 1, 8, 4]*. *b[i] > a[i]* for *i* = 0, 3, and, 4. The array correlation is 3 + 8 + 4 = 15.

Interviewer guidelines

▼ SOLUTION

Skills: Greedy Algorithms, Sorting

Optimal Solution:

We can use a greedy approach here. We will always try to take the maximum values of b as much as possible. We can sort the two arrays in reverse order and iterate through the array *a.* We add *b[j]* to the score and increment *j* at each element where *a[i] < b[j].*

Python Solution:

```
def getMaxArrayCorrelation(a, b):
    list.sort(a, reverse = True)
    list.sort(b, reverse = True)
    ans = 0
    i, j = 0, 0
    n = len(a)
    while i < n and j < n:
        if a[i] < b[j]:
            ans += b[j]
            j += 1
        i += 1
    return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n logn)

Sorting the arrays takes O(n logn) time in the worst case.

Space Complexity - O(n)

Assuming that the inbuilt sorting functions use a variation of merge sort, sorting uses O(n) extra space.

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
```

```java
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {

    /*
     * Complete the 'getMaxArrayCorrelation' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY a
     *  2. INTEGER_ARRAY b
     */

    public static long getMaxArrayCorrelation(List<Integer> a, List<Integer> b) {
        Collections.sort(a);
        Collections.sort(b);

        int i = a.size() -1;
        int j = b.size() -1;
        long sum = 0;
         while (i >= 0 && j >=0){
            if (b.get(j) > a.get(i)){
                sum +=b.get(j);
                i--;
                j--;
            }else{
                i--;
            }
         }

         return sum;



    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int aCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> a = IntStream.range(0, aCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int bCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> b = IntStream.range(0, bCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.getMaxArrayCorrelation(a, b);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

Testcase 0	Easy	Sample	Success	1	0.1056 sec	31.6 KB
Testcase 1	Easy	Sample	Success	1	0.1023 sec	32 KB
Testcase 2	Easy	Sample	Success	1	0.1007 sec	31.8 KB
Testcase 3	Easy	Hidden	Success	3	0.1123 sec	31.8 KB
Testcase 4	Easy	Hidden	Success	3	0.1061 sec	31.1 KB
Testcase 5	Medium	Hidden	Success	4	0.0965 sec	32 KB
Testcase 6	Medium	Hidden	Success	5	0.1273 sec	32.8 KB
Testcase 7	Medium	Hidden	Success	6	0.1381 sec	32.9 KB
Testcase 8	Medium	Hidden	Success	6	0.1502 sec	33.5 KB
Testcase 9	Hard	Hidden	Success	7	0.1682 sec	51.5 KB
Testcase 10	Hard	Hidden	Success	7	0.2677 sec	75.3 KB
Testcase 11	Hard	Hidden	Success	7	0.3853 sec	101 KB
Testcase 12	Hard	Hidden	Success	7	0.4474 sec	128 KB

Testcase 13	Hard	Hidden	Success	8	0.6399 sec	176 KB
Testcase 14	Hard	Hidden	Success	9	0.4423 sec	175 KB

No comments.

7. Java: DataVille's Optimal Storage

Correct

Coding Greedy Sorting Medium Java

Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

Function Description

Complete the function *getOptimalStorage* in the editor below.

Function Parameters

int databaseStorage[n]: the current organization of the database's storage, with 0s for standard data and 1s for priority data

Returns

int: the minimum number of operations required to minimize transitions between 0s and 1s

Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in \{0, 1\}$

INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.

Each of the next lines contains an integer, *databaseStorage[i].*

SAMPLE CASE 0

Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
0
1
```

Sample Output 0

```
1
```

Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
1         databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]
0
0
1
1
0
0
0
1
```

Sample Output 1

```
2
```

Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
```

```
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;

        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }

        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

Time Complexity - O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

Space Complexity - O(1)

We use only 2 variables, hence the space complexity is O(1).

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;

import org.junit.experimental.max.MaxCore;

import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int n = databaseStorage.size();
        int totalOnes =0;
        for(int num: databaseStorage){
            if(num == 1) totalOnes++;
```

```
        }
        int totalzeros =n - totalOnes;
        if(totalOnes == 0 || totalzeros == 0 ){
            return 0;
        }
        int misplacedOnes = 0;

            for(int i = 0; i< totalzeros; i++ ){
                if (databaseStorage.get(i) == 1){
                    misplacedOnes++;
                }
            }
                int misplacedZeros = 0;

                for (int i =0; i < totalOnes;  i++){
                    if(databaseStorage.get(i) == 0){
                    misplacedZeros++;
                }
            }
            return Math.min(misplacedOnes, misplacedZeros);
        }
        }



public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());
```

```

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.106 sec	31.5 KB
TestCase 1	Easy	Sample	Success	1	0.1108 sec	31.1 KB
TestCase 2	Easy	Sample	Success	1	0.1066 sec	31.9 KB
TestCase 3	Easy	Hidden	Success	3	0.1041 sec	31.9 KB
TestCase 4	Easy	Hidden	Success	3	0.1109 sec	31.3 KB
TestCase 5	Easy	Hidden	Success	3	0.1098 sec	31.1 KB
TestCase 6	Easy	Hidden	Success	4	0.0918 sec	31.4 KB
TestCase 7	Easy	Hidden	Success	4	0.1105 sec	31.8 KB
TestCase 8	Easy	Hidden	Success	5	0.1059 sec	31.5 KB
TestCase 9	Easy	Hidden	Success	5	0.1044 sec	31.4 KB
TestCase 10	Easy	Hidden	Success	5	0.1032 sec	31.5 KB
TestCase 11	Medium	Hidden	Success	7	0.134 sec	32.6 KB

TestCase 12	Medium	Hidden	Success	8	0.1221 sec	32.6 KB
TestCase 13	Hard	Hidden	Success	10	0.1251 sec	32.9 KB
TestCase 14	Hard	Hidden	Success	15	0.3146 sec	102 KB

No comments.